Language, Attention and Transformers

Lecture 11 – Introduction to Modern Machine Learning

Steven Campbell

Imperial College London Summer School

June 2026

Outline

1. Text as data
2. Attention
3. Transformers
4. Putting it together

Where this lecture fits

Earlier lectures introduced:

- loss functions, validation, regularisation and overfitting;
- dimension reduction and learned representations;
- online learning and sequential feedback;
- multilayer perceptrons and convolutional neural networks.

Today

We ask: how do neural networks process **text**? We move from tokens and embeddings to attention and Transformer blocks.

This will help us understand large language models and generative AI.

Scope

We will not train a large language model. The goal is to understand the core mechanism that makes modern language models possible.

Text as data

Why text is different from images

In Lecture 10, images were arrays of numbers:

$$\text{image} \in \mathbb{R}^{C \times H \times W}.$$

Text is different:

- raw text appears as discrete symbols, not continuous pixel intensities;
- the order of the text changes the meaning (e.g., “dog bites man” differs from “man bites dog”);
- meaning is contextual: “bank” can mean a financial institution or a river edge;
- sentences have variable length;
- punctuation can change meaning.

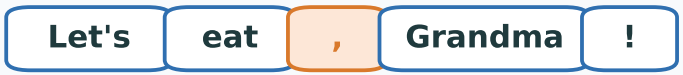
Commas save lives

Tiny symbols can change sentence structure and meaning.

Without comma

Grandma is part of what is eaten

With comma



Grandma is being addressed

For NLP, punctuation and order are part of the signal.
Tokenisation should keep enough of that structure for the task.

Core challenge

Turn discrete text into vectors while preserving enough order and context for learning.

From text to model inputs

Neural networks do not read text directly. Rather, they first convert it into numerical vectors.

The hard part is preserving enough order and context for the task.

Tokens and tokenisation

A **token** is a piece of text used as a unit by the model.

Examples:

- word tokens: `[online, learning, is, useful]`;
- character tokens: `[o, n, l, i, n, e, ...]`;
- subword tokens: `[re, train, ing]`.

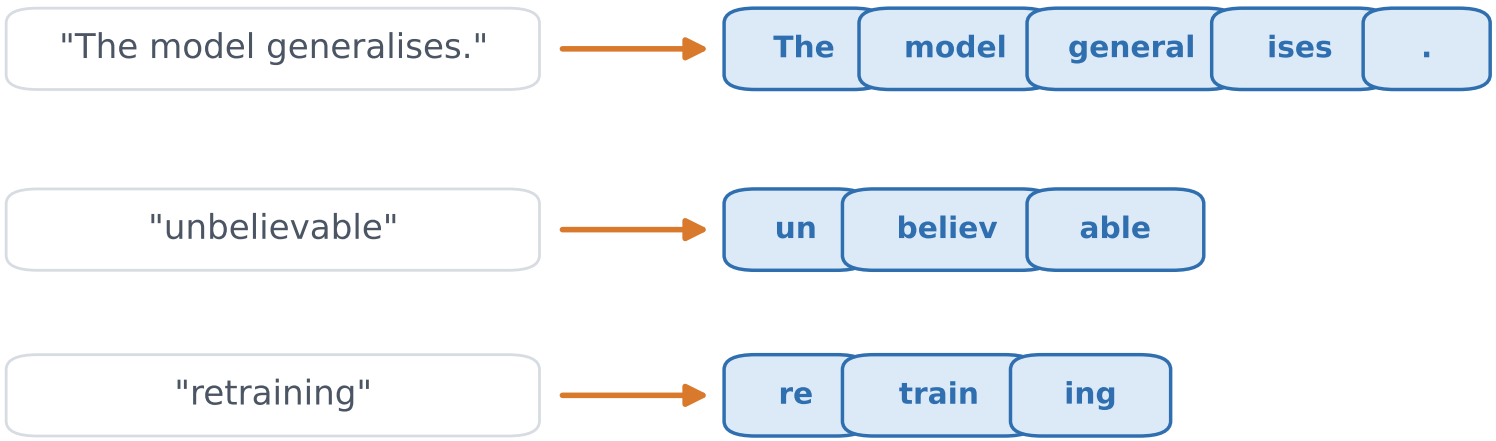
Why subwords?

Whole words create huge vocabularies and cannot handle new words well. Characters are flexible but create long sequences. Subwords are a practical compromise.

In practice the tokenisation of text also has to handle punctuation, capitalisation, Unicode and special characters.

Subword tokenisation examples

Tokenisation breaks text into model-readable pieces



Subwords are a practical compromise: shorter than whole-word vocabularies, longer than characters.

Embeddings: learned vectors for tokens

After tokenisation, each token is mapped to an integer ID. An embedding layer stores a vector for each token:

$$E \in \mathbb{R}^{V \times d},$$

where V is the vocabulary size[1] and d is the number of coordinates used to represent each token.

If token i appears, the embedding layer performs an **embedding lookup**:

$$e_i = E_{i,:} \in \mathbb{R}^d.$$

Connection to representation learning

Embeddings are learned coordinates for discrete objects. Unlike a fixed projection such as PCA, they start random and are **learned** as the model trains.

[1] GPT 3 had a vocabulary size of 50,257 tokens.

How are embeddings learned?

Nobody hand-picks the numbers in E: they start out **random** and are **learned automatically** as the model trains. For general word embeddings the task is usually **self-supervised**. That is, models predict a word from its **neighbours**, using only raw text (no labels).

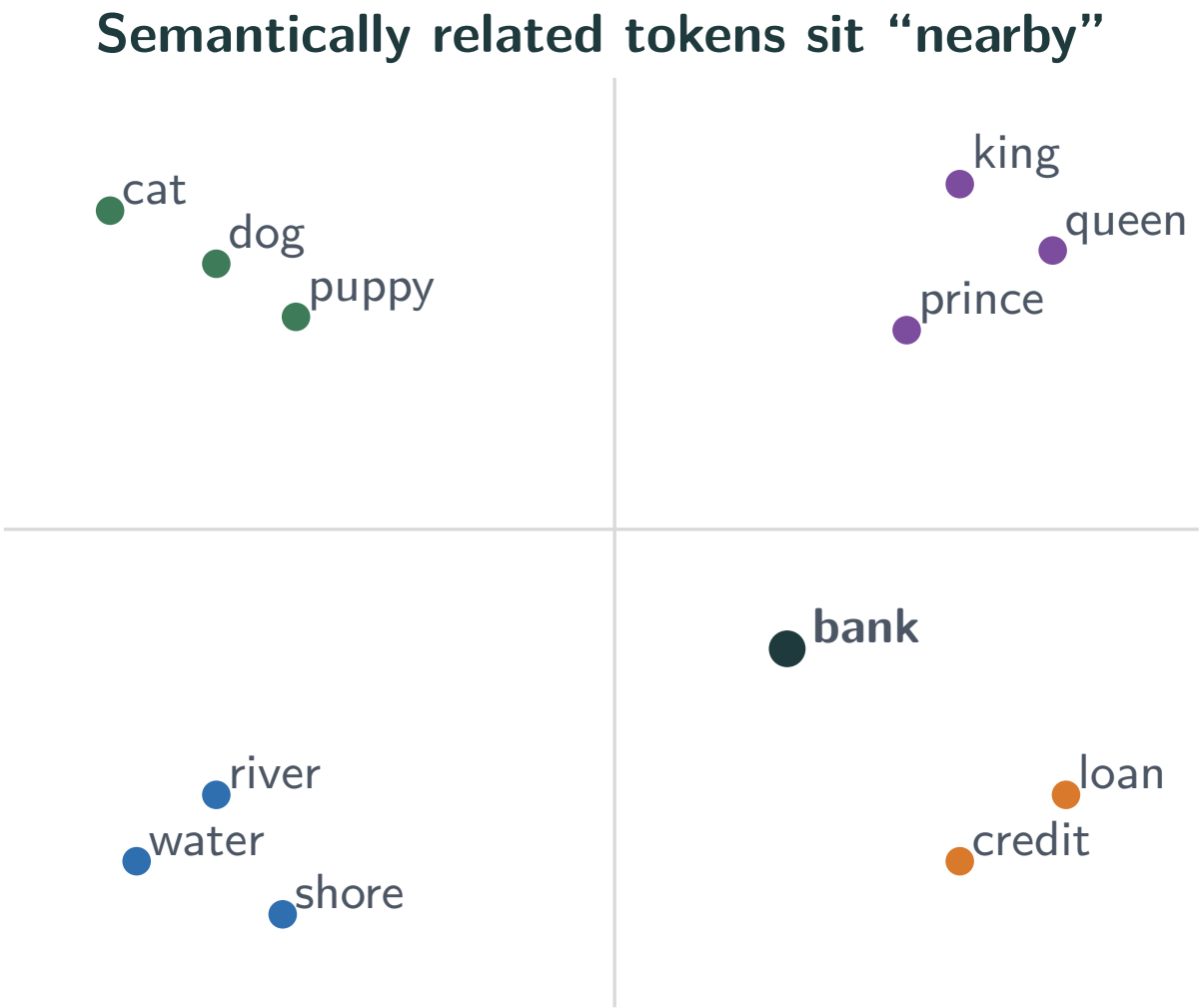
1. the model uses the surrounding word vectors (the context) to **predict** a missing word;
2. it measures how wrong it was (the **loss**);
3. gradient descent **nudges** each vector to make the loss smaller;
4. repeat for millions of examples.

Why this produces meaning

"You shall know a word by the company it keeps." —J.R. Firth

Tokens in **similar contexts** get nudged towards **similar vectors**, so related ones move together and unrelated ones move apart.

A toy embedding space

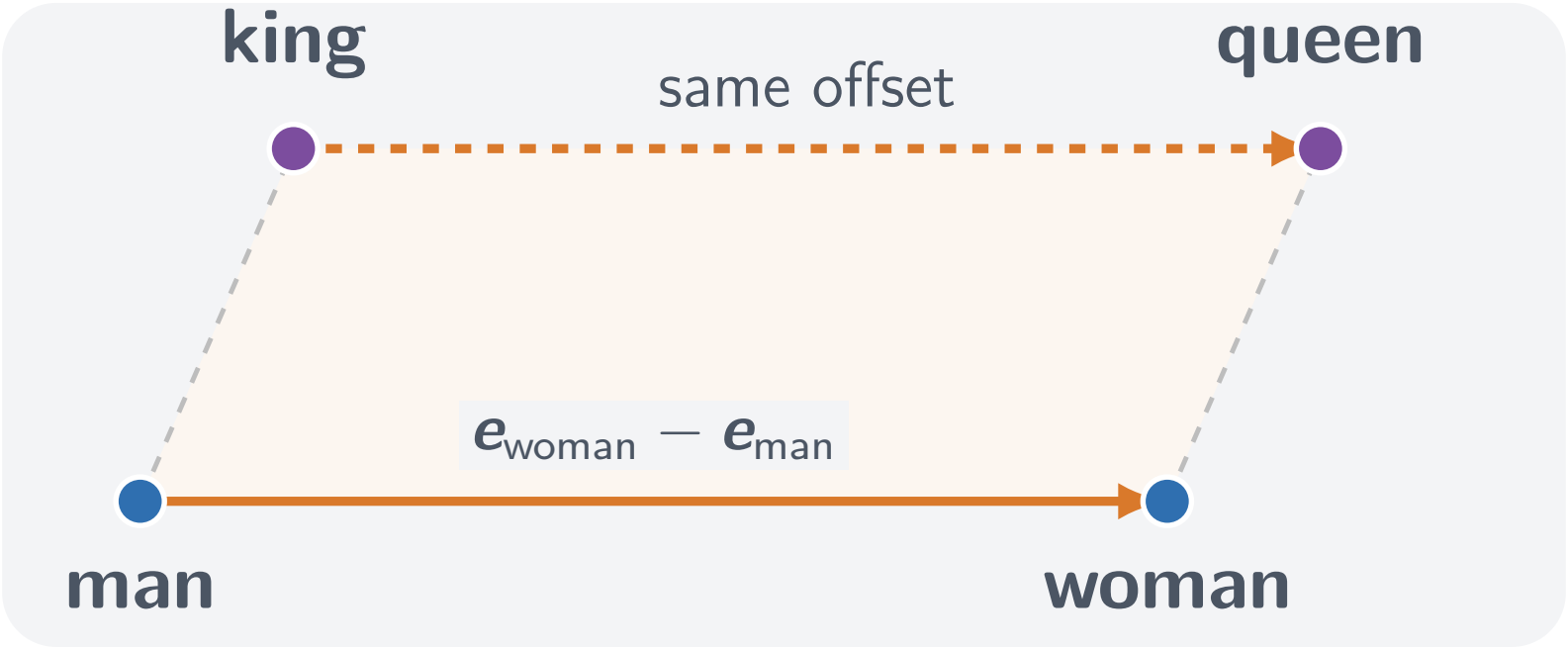


Real embeddings have hundreds or thousands of dimensions.[2]

[2] GPT 3 used an embedding of size 12,288.

A famous learned relationship

$$\boldsymbol{e}_{\text{king}} + (\boldsymbol{e}_{\text{woman}} - \boldsymbol{e}_{\text{man}}) \approx \boldsymbol{e}_{\text{queen}}$$

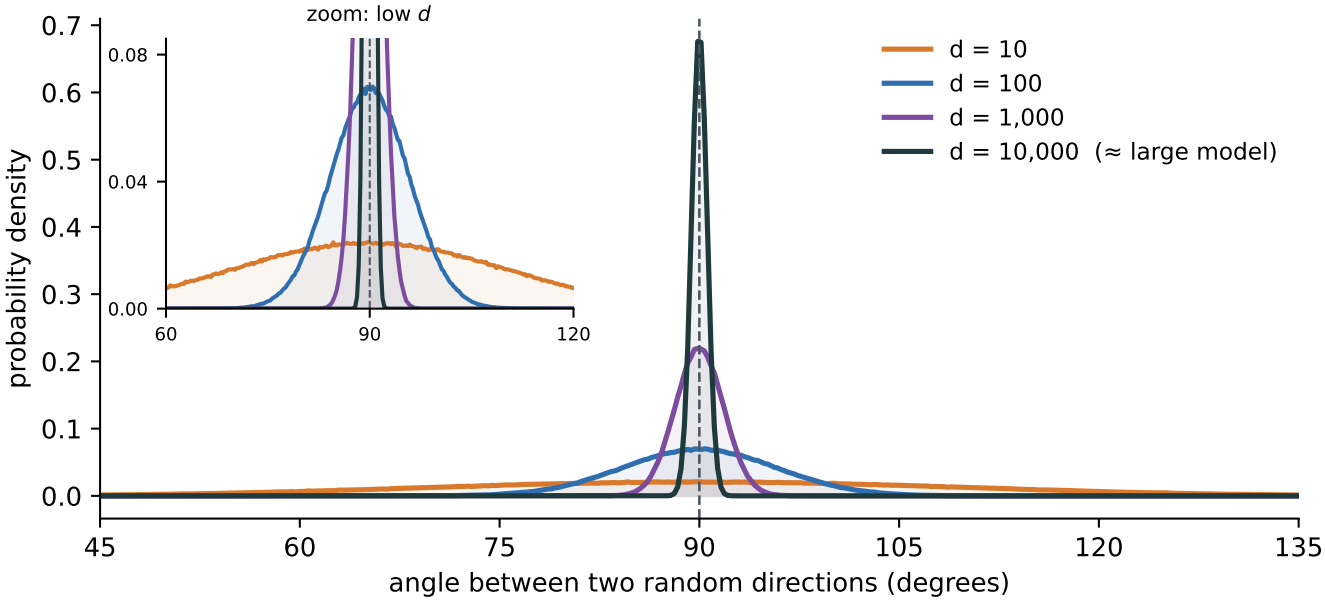


This is a stylized 2D sketch. Offsets are approximate, high dimensional, and can reflect bias.

Consistent **directions** encode relationships in general (e.g. country $\rightarrow$ capital, or singular $\rightarrow$ plural).

Why so many dimensions?

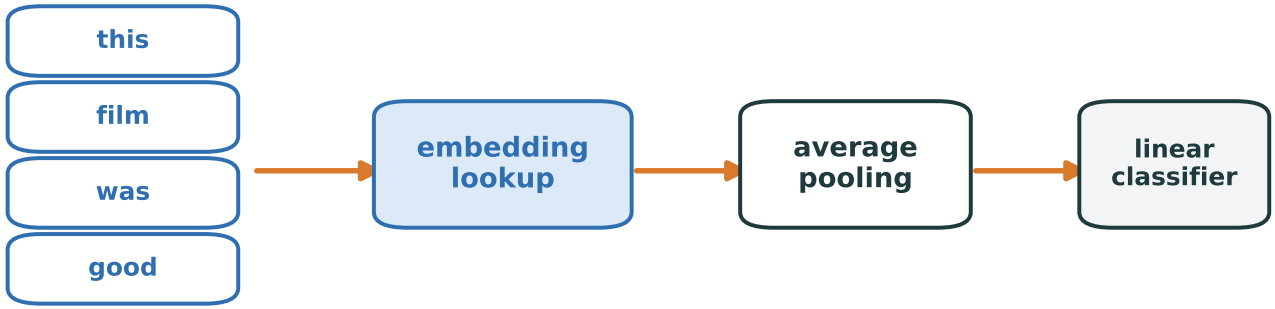
Our embeddings have **thousands** of dimensions. Here is a surprising reason that helps.



The number of "**nearly perpendicular**" vectors scales roughly **exponentially** in the dimension; i.e., there are far more such vectors than the dimensions themselves. A model can therefore encode many more concepts than it has dimensions (a concept called **superposition** which is a leading idea for why scaling models helps).

Using embeddings: a simple classifier

Once embeddings are **pre-trained** on raw text we can **reuse** them as fixed features for a labeled task.[3] For instance, assessing **sentiment** in a text passage.



A simple baseline: average token embeddings, then apply a classifier.

Useful, but it loses word order and does not adapt meanings to context.

Limitation

Averaging is simple but has two gaps: it **ignores word order**, and gives each token the **same vector in every context**.

[3] Once the embeddings are given, only the classifier on top needs to be trained.

The key idea before attention

Suppose the token "bank" appears in two contexts:

"the bank approved the loan"

"the river bank flooded".

The embedding lookup gives the **same starting vector** to "bank" in both, but the two should **mean** different things!

Goal

Create **contextual representations**: vectors that depend on the surrounding tokens, computed separately for each context window.[a]

[a] e.g., a given sentence.

Attention is the mechanism that does this: it lets each token's representation draw on the other tokens around it.

Static embeddings vs contextual representations

Embedding lookup

$$\text{bank} \mapsto e_{\text{bank}}$$

same starting vector
for every occurrence

After attention layers

river bank

bank loan

$$h_{\text{river bank}} \neq h_{\text{bank loan}}$$

Attention turns context-independent token vectors into context-dependent representations.

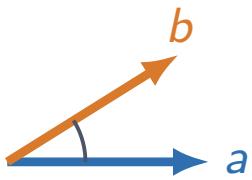
Attention

Measuring token similarity

Two vectors' **dot product** tells you how much they point the same way:

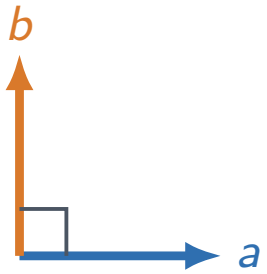
$$a \cdot b = \sum_i a_i b_i = \|a\| \, \|b\| \cos\theta,$$

so its **sign** is set by the angle θ between them.



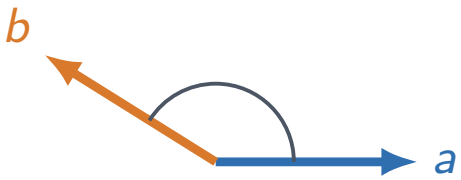
nearly aligned

$a \cdot b > 0$



perpendicular

$a \cdot b = 0$



nearly opposed

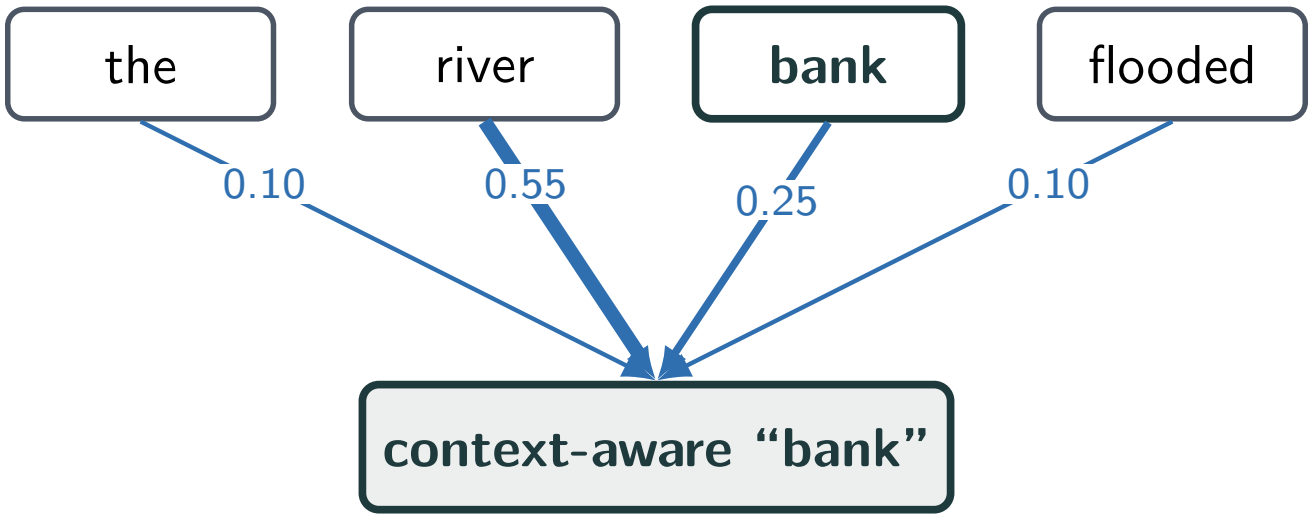
$a \cdot b < 0$

One way to assess how **related** one token is to another is by taking the dot product of their vectors.

What attention does

Attention

Each token builds a new vector by taking a **weighted average** of "information" from the other tokens.

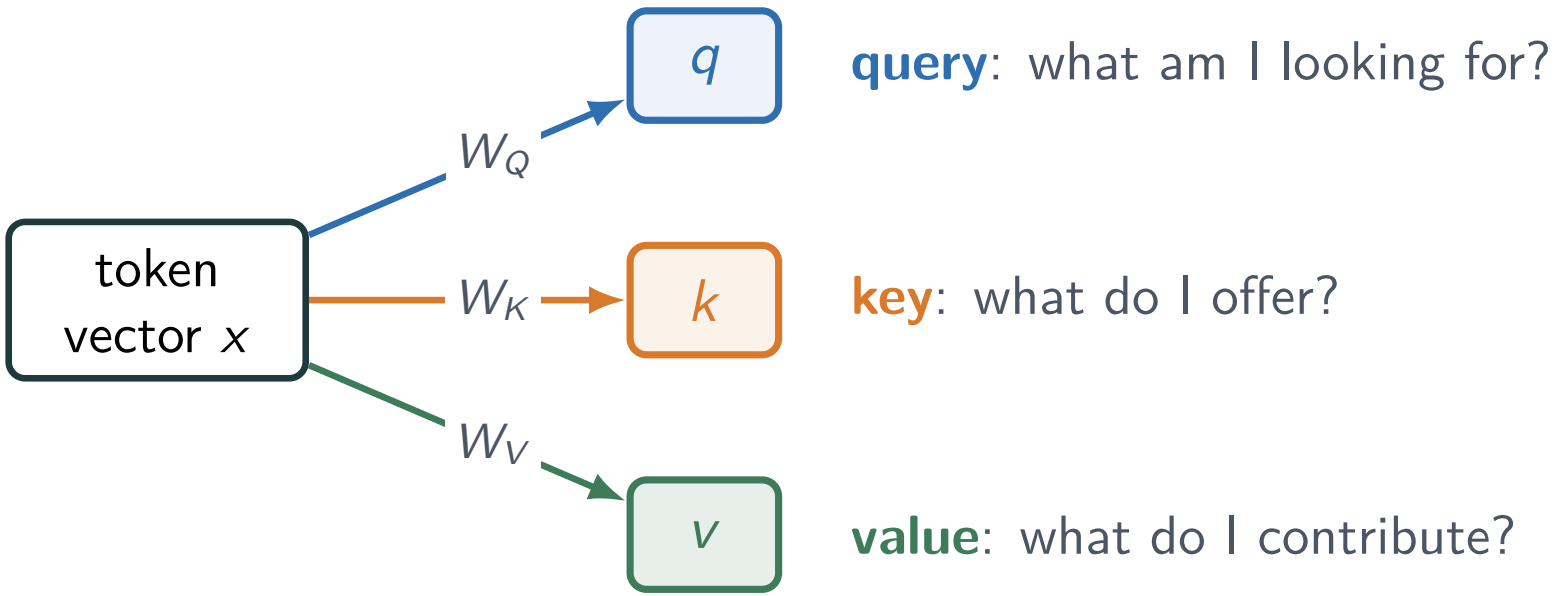


Loosely, the weighted contributions depend on each token's **relevance**.

Queries, keys and values

Each token's current vector x is turned into three vectors by three **learned** matrices:

$$q = x\,W_Q, \qquad k = x\,W_K, \qquad v = x\,W_V,$$



Every token produces its own q, k, v. Queries and keys get **compared**; values carry the **information** mixed into the result.

Putting them together

For the token being updated, compare its query against **every** key, normalise the scores, then average the values:

$$\underbrace{s_i = q \cdot k_i}_{\text{relevance}} \longrightarrow \underbrace{\alpha = \text{softmax}(s)}_{\text{weights, sum to 1}} \longrightarrow \underbrace{\textstyle\sum_i \alpha_i \, v_i}_{\text{new vector}}$$

These α_i are the arrow weights from our first illustration of attention. The query meets each key, and the closer they point, the larger that token's share. Because q, k, v come from the **current** vectors, tokens attend to one another differently in different sentences.

One step of attention, with numbers

We update **bank** in *"the river bank flooded"*: its query q meets every key, and **softmax** turns the scores into weights.

token	**score** $q \cdot k_i$	**weight** $\alpha_i = \mathrm{softmax}(s)_i$
the	0.5	**0.10**
river	2.2	**0.55**
bank	1.4	**0.25**
flooded	0.5	**0.10**

$$v_{\text{bank}}^{\text{new}} = \sum_i \alpha_i v_i = 0.10\, v_{\text{the}} + 0.55\, v_{\text{river}} + 0.25\, v_{\text{bank}} + 0.10\, v_{\text{flooded}}$$

We see that **river** dominates the average, so the bank vector shifts toward a vector that reflects the "river bank" meaning.

Softmax attention continued

Earlier we scored relevance with $q \cdot k_i$. In practice we use the **scaled** dot product,

$$s_i = \frac{q \cdot k_i}{\sqrt{d_k}},$$

where d_k is the dimension of the query and key vectors. The weights and output are

$$\alpha_i = \frac{\exp(s_i)}{\sum_j \exp(s_j)}, \qquad z = \sum_i \alpha_i \, v_i.$$

Why these choices

Softmax turns scores into weights, with larger similarity receiving a larger weight. Dividing by $\sqrt{d_k}$ stops the scores from growing with dimension, which keeps the softmax gradients from saturating.

The **rule** (W_Q, W_K, W_V) is learned once while the weights α_i are computed afresh from the **current** input.

Self-attention

So far we updated *one* token. In **self-attention**, *every* token is updated at once, with queries, keys and values all drawn from the **same** sequence.

Stack the T tokens as the rows of $X \in \mathbb{R}^{T \times d}$, where d is the representation dimension. The same learned maps, applied to every row, give

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

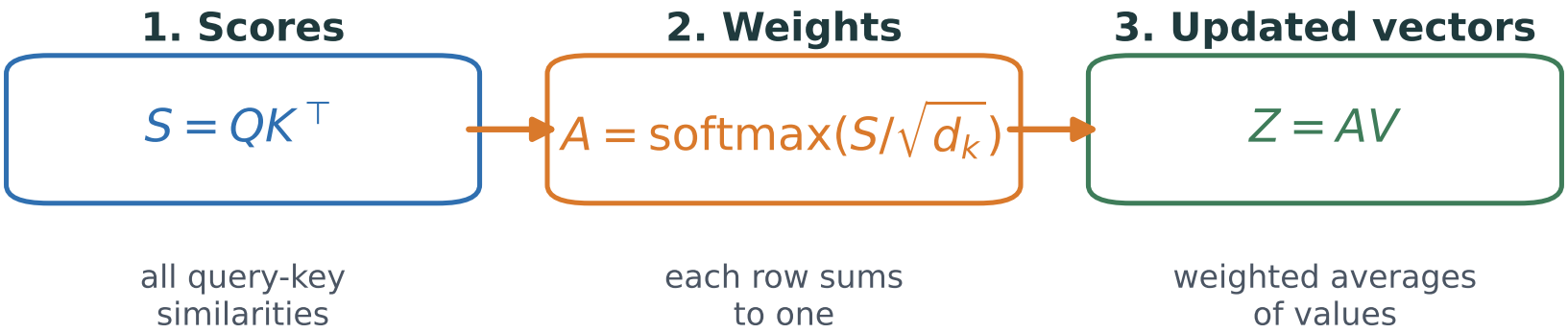
Row i holds token i's vectors — $q_i = x_i W_Q$, and likewise k_i and v_i.

Key point

Every token updates its representation by attending to all tokens in the same sequence, which can be computed for the whole sequence in parallel.

Self-attention in matrix form

Self-attention as three matrix operations

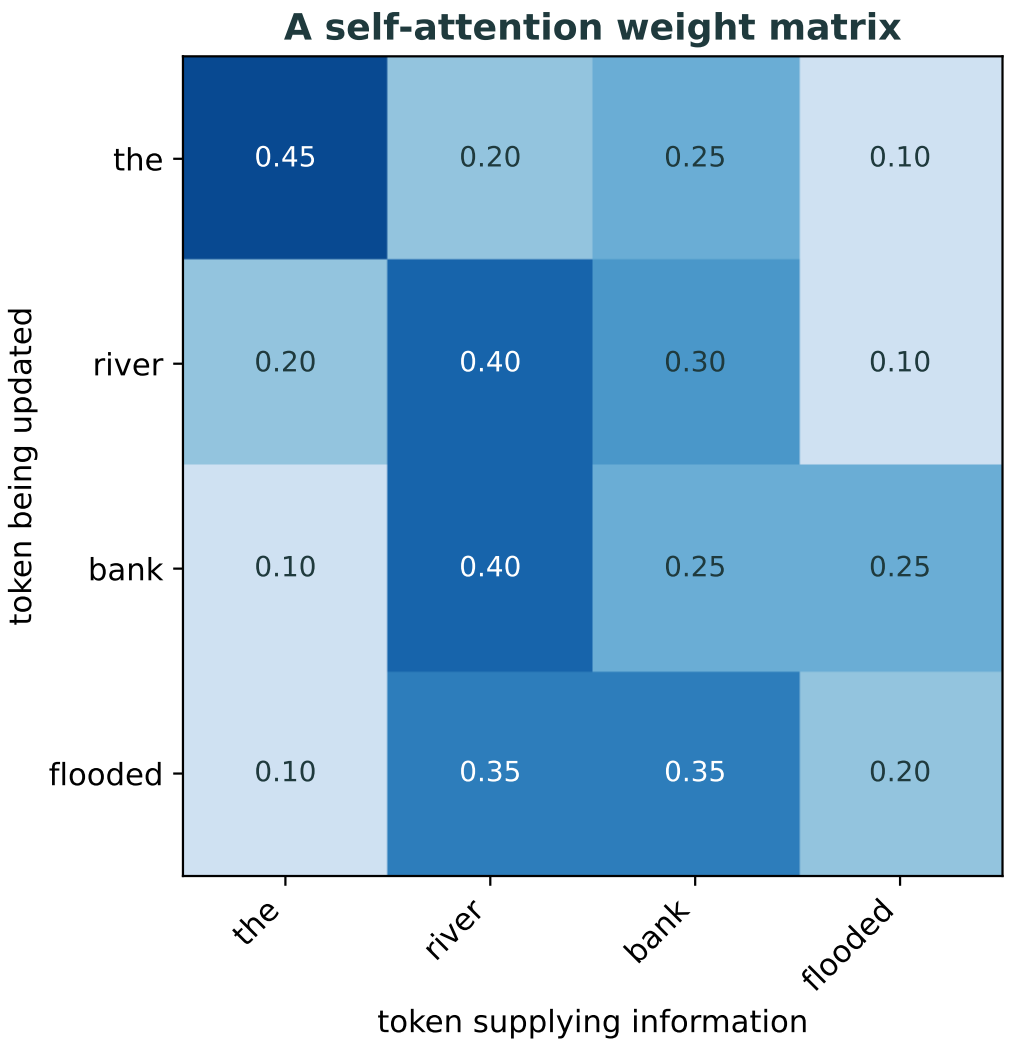


In the compact formula: $\mathrm{Attention}(Q, K, V) = \mathrm{softmax}(QK^\top/\sqrt{d_k})V$.

Interpretation

$S = QK^\top$ holds every query–key score, $A = \mathrm{softmax}(S/\sqrt{d_k})$ turns each row into weights, and $Z = AV$ stacks the updated token representations.

A self-attention weight matrix

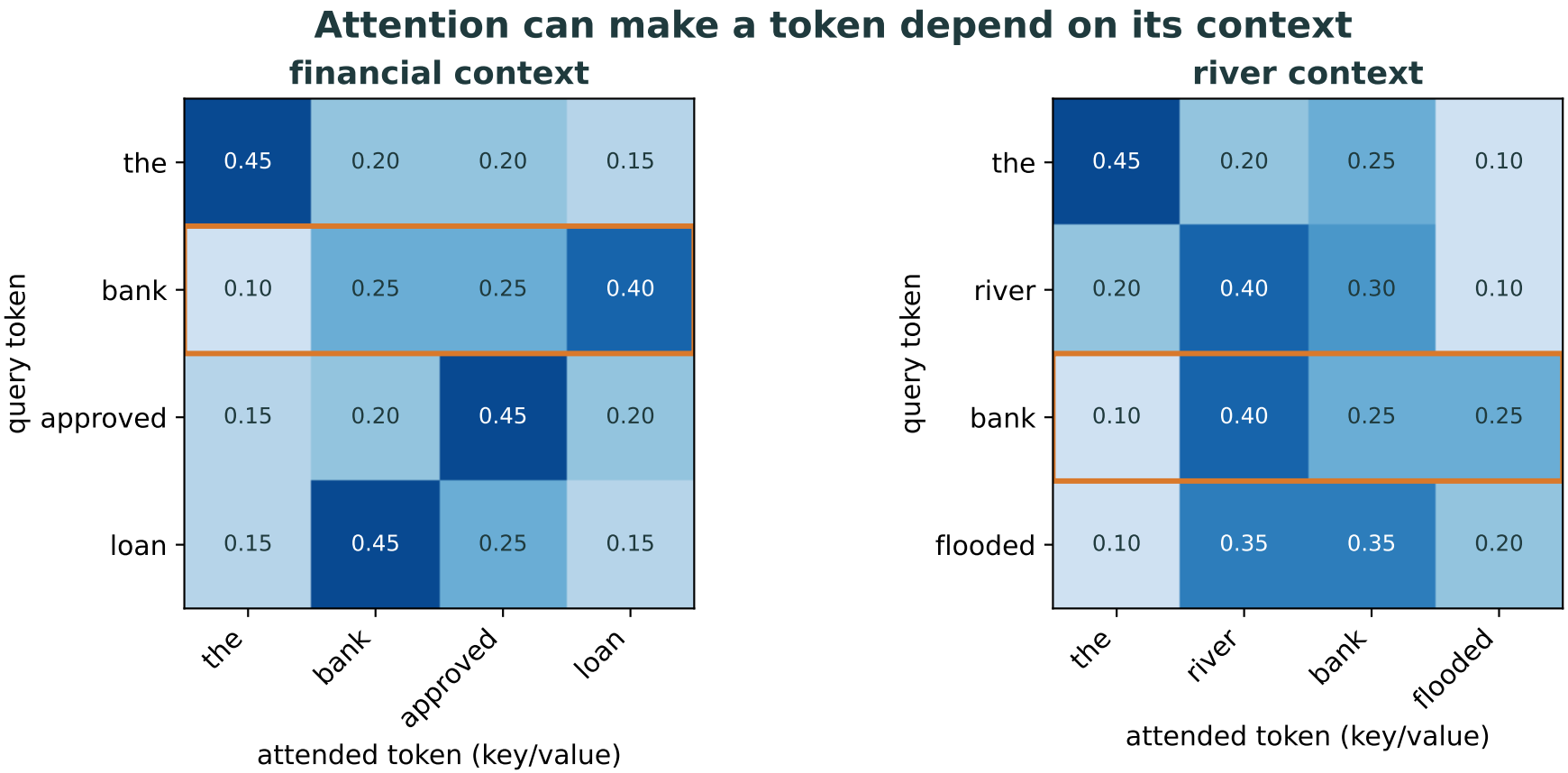


Reading a row

Row i shows how much each token contributes to the updated representation of token i.

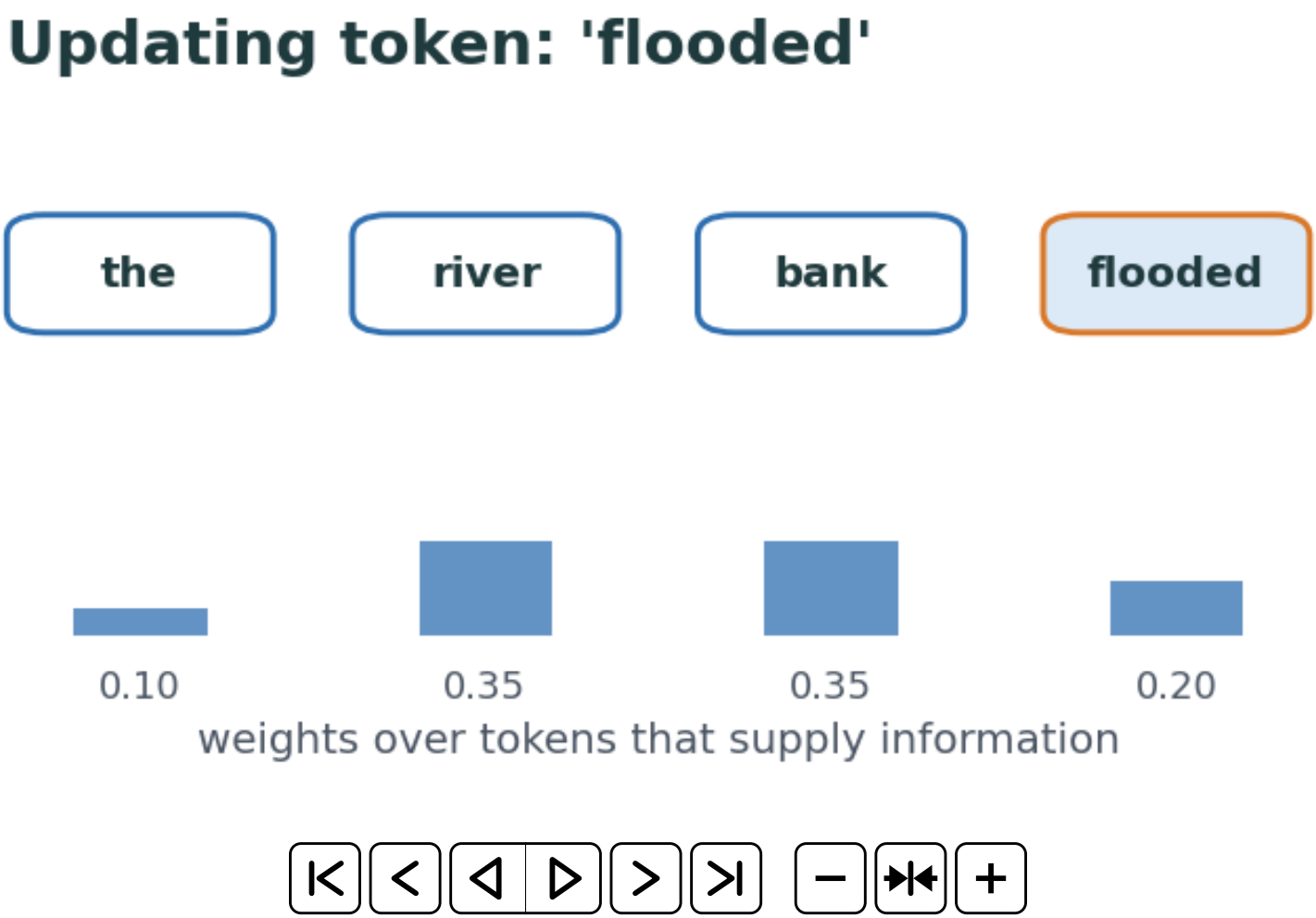
Each row is a **softmax** output, so its entries are nonnegative and sum to one.

Different contexts visualised



The same token attends to different words in different sentences, so it ends up with a different representation.

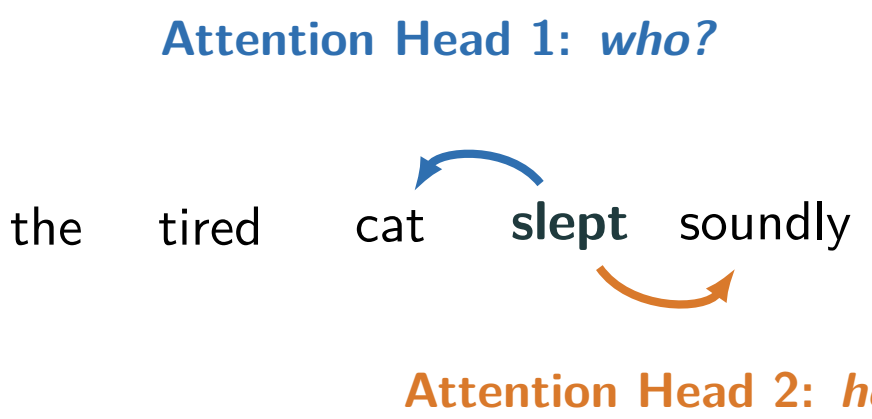
Self-attention: animated view



Each row of the attention matrix updates one token as a weighted average of the other token representations.

One pattern is not enough

A single attention layer gives each token *one* set of weights. This represents **one way** of combining the meanings of each token. But a word often relates to several others, for different reasons.



The same word needs its **subject** *and* its **adverb**. One self-attention operation likely cannot capture both at once ⇝ we can run several attentions in **parallel**.

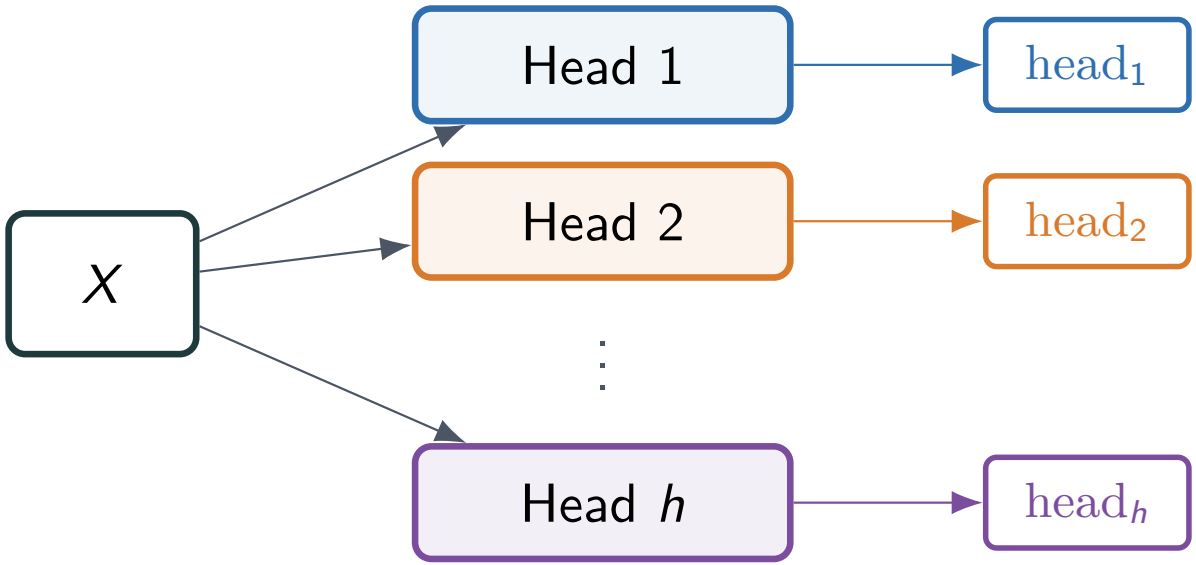
Multi-head attention

Run h attention **heads** in parallel. Each has its *own* learned projections and computes the self-attention you already know:

$$Q_\ell = XW_Q^{(\ell)}, \qquad K_\ell = XW_K^{(\ell)}, \qquad V_\ell = XW_V^{(\ell)},$$

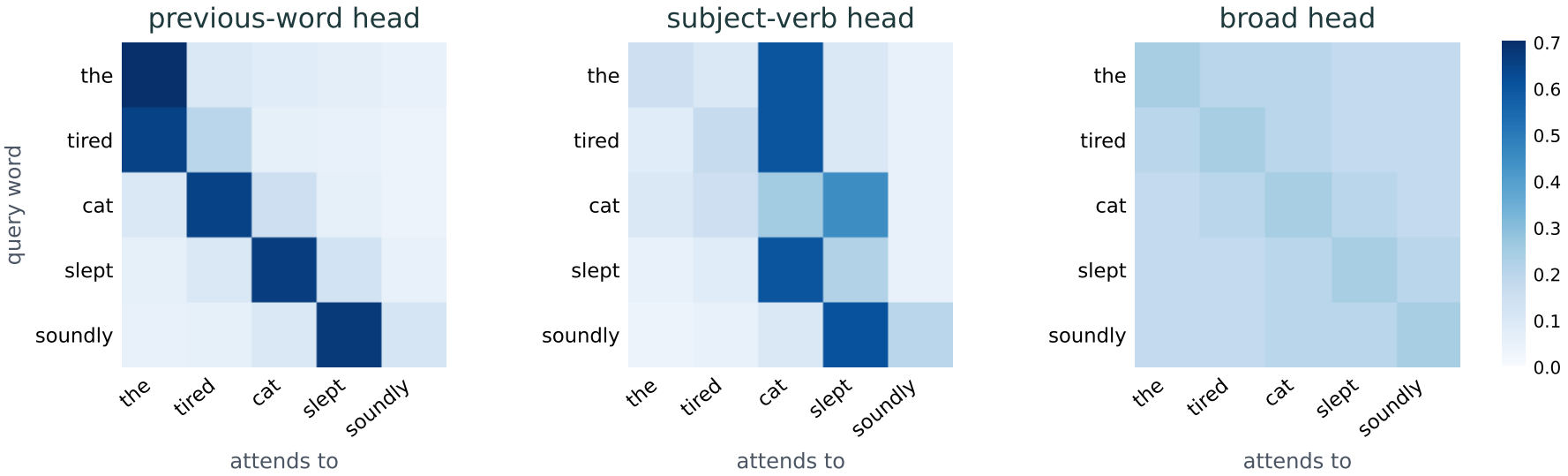
$$\text{head}_\ell = \text{softmax}\left(\frac{Q_\ell K_\ell^\top}{\sqrt{d_k}}\right) V_\ell.$$

Each head works in a **smaller** space ($d_k = d_v = d/h$), so all h heads together cost about the same as one full-size attention.



What the heads learn

As with the CNN filters, no one tells the heads what to specialise in. The structure is **learned**. In a trained model, different heads pick up different relationships:

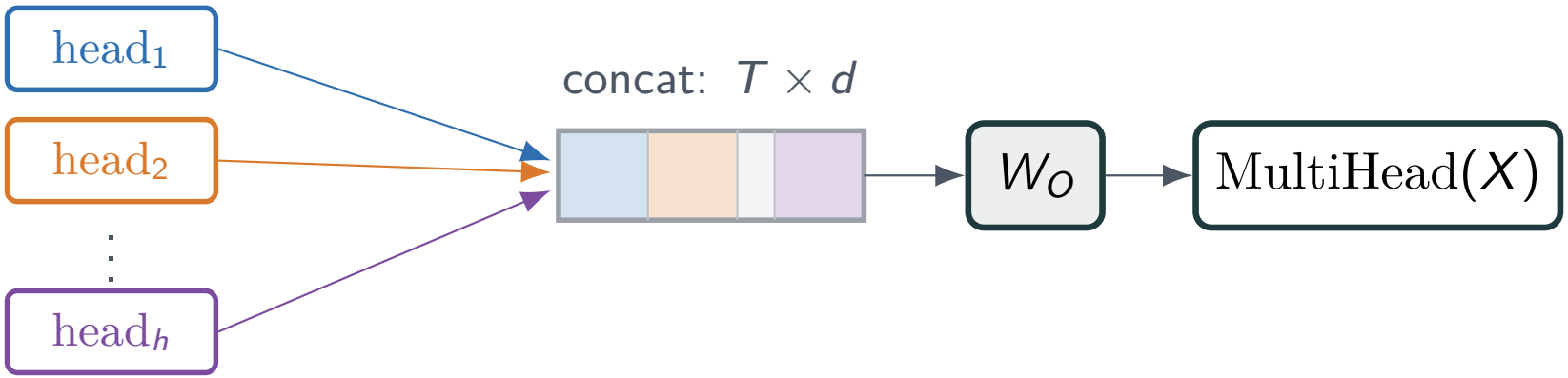


Some heads track the **previous word**, some link **subject and verb**, some spread attention broadly. Each token ends up with several **complementary** views of its context.

Combining the heads

Stack the head outputs side by side and pass them through one more **learned** projection W_O:

$$\mathrm{MultiHead}(X) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\, W_O.$$



Each head is $T \times d/h$, so the concatenation is $T \times d$ again. W_O **mixes** the heads and returns an output the same shape as X.

Transformers

Attention alone does not know word order

Self-attention compares tokens to other tokens, but without extra information it has no built-in notion of first, second or third.

For example, recall that these contain the same words but describe different events:

$$\text{dog bites man}, \qquad \text{man bites dog}.$$

Solution

Add positional information to token embeddings:

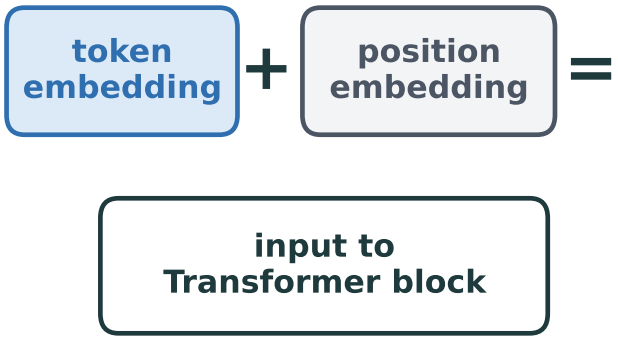
$$\text{input representation} = \text{token embedding} + \text{position embedding}.$$

The model can then use both **what** a token is and **where** it appears.

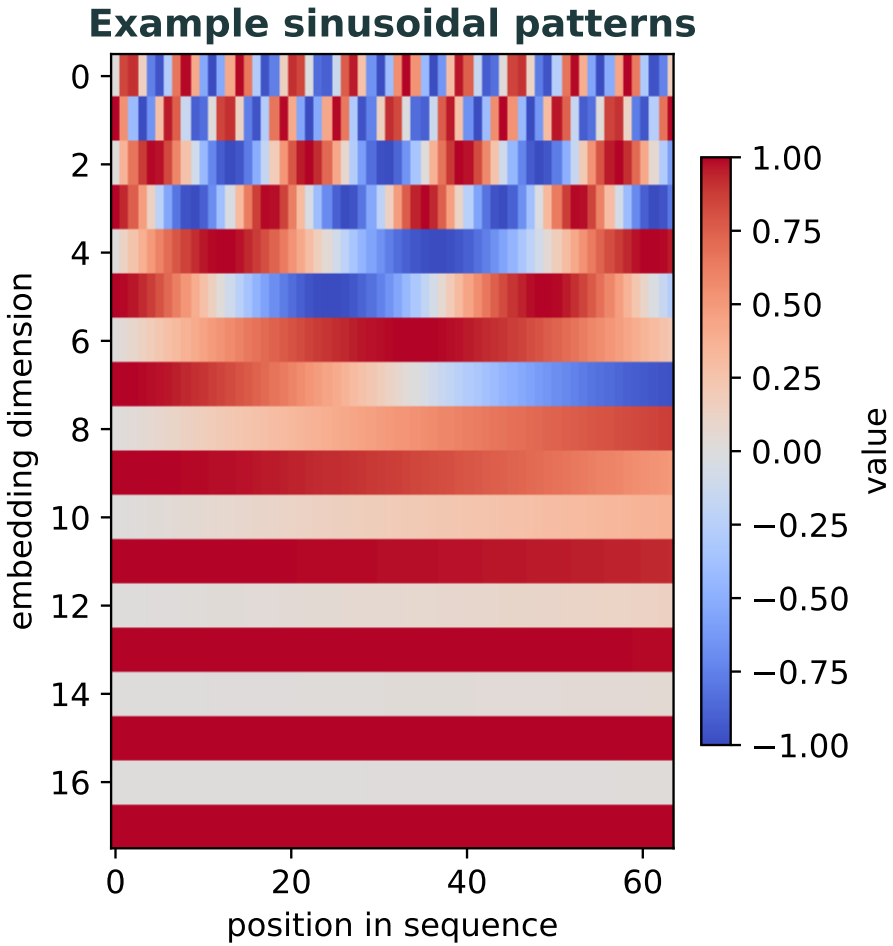
More precisely, self-attention without positional information is permutation equivariant: if we shuffle the tokens, the outputs shuffle in the same way.

Positional encodings

Add position information

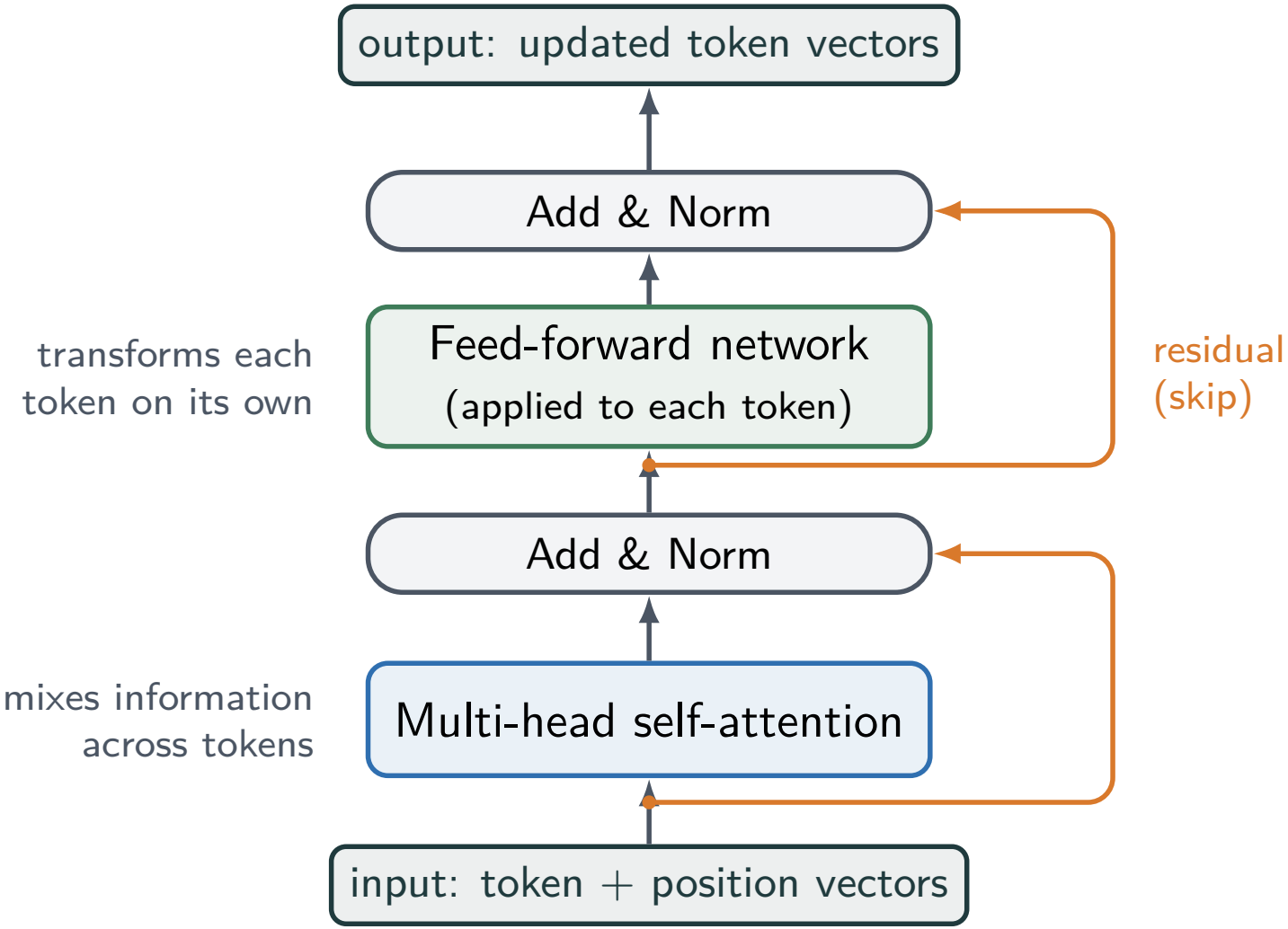


Without positions, attention can compare tokens but does not know which token came first.



Lower dimensions oscillate quickly and higher dimensions slowly, giving each position a distinct pattern. Many models instead learn position embeddings; sinusoidal encodings are one common explicit construction.

A Transformer block



Shown in *post-norm* form (normalisation after each sub-layer). Many modern models use *pre-norm*, normalising *before* each sub-layer for more stable training.

What do the components do?

- **Multi-head self-attention:** lets tokens exchange information in several ways in parallel.
- **Feed-forward network:** a small neural network applied to **each token on its own**; i.e., the same network is applied at every position, with no information passing between tokens.
- **Residual connections:** add the input of a sublayer back to its output, helping information and gradients flow.
- **Normalisation:** stabilises the scale of activations during training.

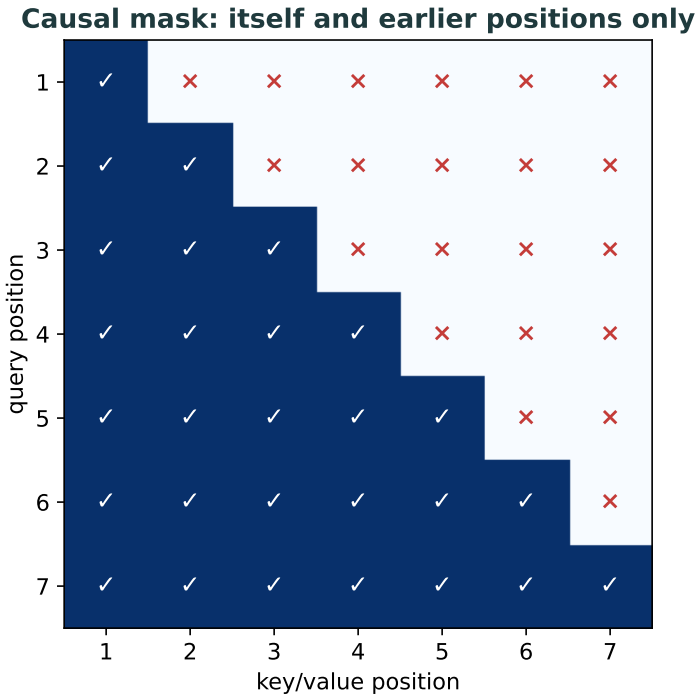
Two complementary effects: attention *mixes* information *across* tokens; the feed-forward network *transforms each token*.

Stacking

A Transformer stacks many such blocks, each with its *own* weights. Every block refines the token representations, so deeper blocks build on the features formed by earlier ones.

Causal masking for generation

For text generation, token t should not see future tokens $t+1, t+2, \ldots$; it may attend only to itself and earlier positions.



Why?

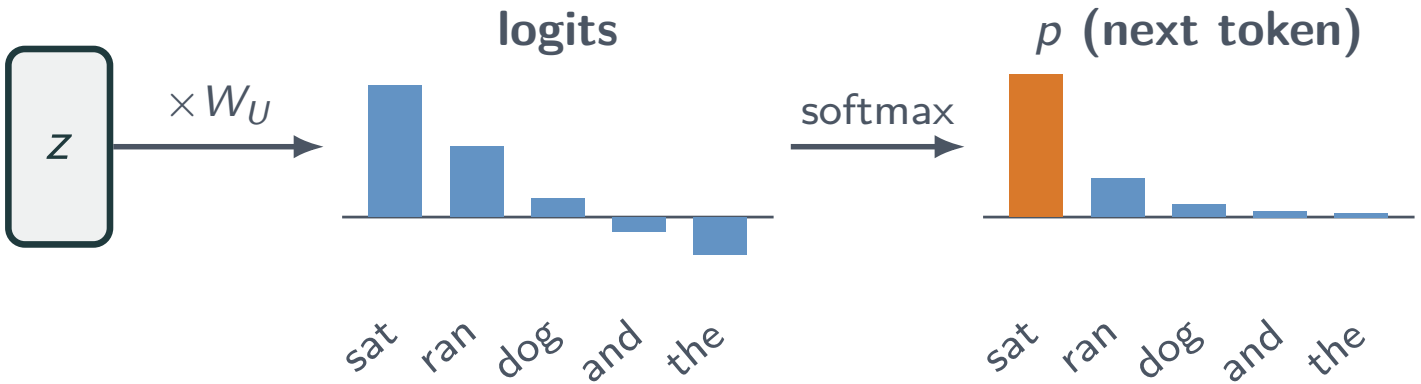
A model trained for generation must predict the next token using only previous tokens, just as it will at test time.

From vectors to a next-token distribution

After the final block, every position holds a context-rich vector. The model turns the *last* token's vector $z \in \mathbb{R}^d$ into a prediction for the next token: score the whole vocabulary, then normalise.

$$\text{logits} = z\,W_U \in \mathbb{R}^{|V|}, \qquad p = \text{softmax}(\text{logits}),$$

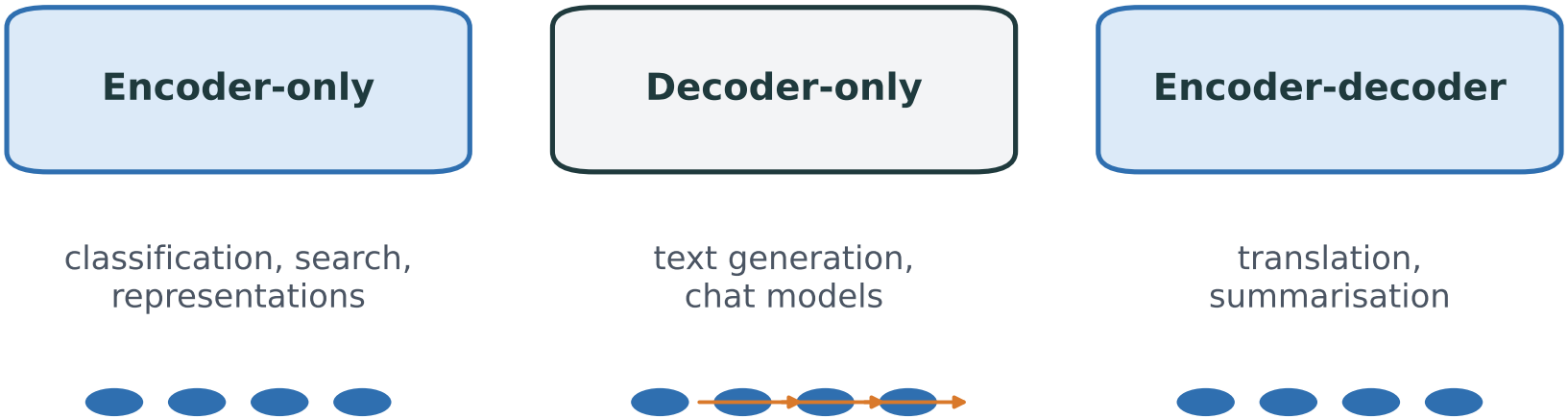
with **unembedding** $W_U \in \mathbb{R}^{d \times |V|}$ and vocabulary size $|V|$.



p is a **probability distribution over the next token**.

Transformer model families

Transformer families are used differently



For Lecture 12, the key case is decoder-only autoregressive generation.

Encoder-only, decoder-only, encoder-decoder

Encoder-only

Sees: whole input sequence, including left and right context.
Used for: classification, embeddings, search.

Decoder-only

Sees: previous tokens only, through a causal mask.
Used for: language generation and chat models.

Encoder-decoder

Sees: encoder reads input; decoder generates output.
Used for: translation and summarisation.

Bridge to Lecture 12

Large language models used for generation are often decoder-only Transformers trained to predict the next token.

Putting it together

What is learned?

During training, the model learns many matrices:

- token embeddings E;
- query, key and value projections W_Q, W_K, W_V: queries and keys produce the input-dependent attention weights; values carry the information those weights combine;
- the output projection W_O that mixes the heads;
- feed-forward network weights;
- the unembedding W_U, which turns the final token vector into next-token scores.

Representation view

Each Transformer layer converts token vectors into more contextual token vectors. The attention weights are recomputed for each input, so the representation of a token can depend on the full available context.

This is why the same word can be represented differently in different sentences.

What we have not covered

There are important details that we are deliberately leaving for other more advanced courses:

- efficient tokenisation algorithms;
- training large Transformers at scale;
- pre-Transformer history like recurrent networks, GRUs and LSTMs;
- optimisation tricks and distributed training;
- mathematical analysis of attention mechanisms.

Summary

- Text must be tokenised and converted into vectors before it can enter a neural network.
- Embeddings are learned starting vectors for tokens.
- Fixed embeddings are not enough: order and context change meaning.
- Attention computes input-dependent weighted averages of value vectors using query-key similarity.
- Self-attention lets tokens update their representations using other tokens in the same sequence.
- Positional information is needed because attention alone does not encode order.
- Transformers stack self-attention and feed-forward blocks to build contextual representations.
- A final unembedding step turns the last token's final vector into a distribution over the next token.